

EconoCausal — Independent 15-Day Sprint Plan

1. Sprint Objective

Build a complete demo-ready EconoCausal platform in 15 days where all five team members can work **independently and in parallel**.

The final system will contain:

- Synthetic customer dataset
- PostgreSQL database
- Causal ML pipeline
- ITE estimation
- Uplift analytics
- Qini/AUUC metrics
- Budget optimization
- Customer recommendations
- FastAPI backend
- Next.js dashboard
- Docker deployment
- End-to-end demo

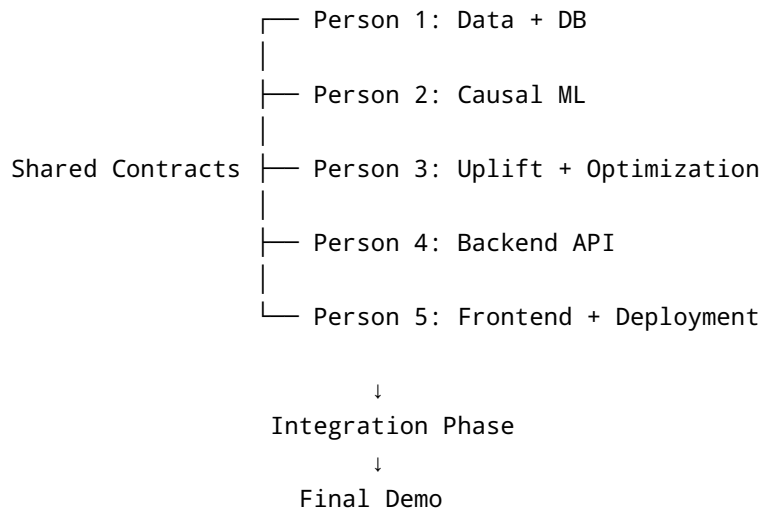
The original plan assigns five areas: Data + Database, Causal ML, Uplift + Optimization, Backend API, and Frontend + Deployment. This plan keeps those ownership areas but removes runtime and development dependencies between them.

2. New Independent Development Strategy

Instead of:

```
graph TD; P1[Person 1] --> P2[Person 2]; P2 --> P3[Person 3]; P3 --> P4[Person 4]; P4 --> P5[Person 5];
```

use:



Everyone develops against **predefined contracts, schemas and mock/sample data**.

3. Day 1 — Common Contract Freeze

This is the only activity that must be coordinated by the entire team.

Common artifacts to freeze

Dataset contract

Define a standard customer dataset containing fields such as:

```
customer_id
age
gender
income
tenure
purchase_frequency
avg_order_value
previous_purchases
engagement_score
treatment
discount
conversion
revenue
cost
```

Causal output contract

Person 2 must produce:

```
{
  "customer_id": "C001",
  "baseline_probability": 0.32,
  "treatment_probability": 0.51,
  "ite": 0.19
}
```

Uplift output contract

Person 3 works with:

```
{
  "customer_id": "C001",
  "ite": 0.19,
  "uplift_segment": "persuadable",
  "expected_conversion": 0.51,
  "recommended_discount": 15,
  "expected_profit": 120,
  "expected_cost": 15,
  "roi": 7.0
}
```

Backend API contract

Person 4 defines the API response structures independently.

Example:

```
GET /health
GET /customers
GET /summary
GET /causal-summary
GET /ite
GET /uplift
GET /recommendations
GET /optimize
```

The original plan already defines these endpoints.

Frontend contract

Person 5 creates the UI using **mock JSON matching the API contract**.

Therefore Person 5 does not need the backend to exist.

4. Person 1 — Data + Database

Objective

Build a reliable and reproducible data foundation.

The original responsibility includes PostgreSQL tables, loading scripts, validation, indexes and summary views.

Days 1-3 — Database Foundation

Day 1

- Review/freeze dataset schema
- Create PostgreSQL database
- Create database schema
- Create tables:
 - customers
 - campaigns
 - model_runs
 - predictions
 - recommendations

Day 2

- Create seed scripts
- Create CSV/JSON import script
- Load synthetic data
- Add primary/foreign keys
- Add basic indexes

Day 3

- Create validation scripts
- Validate:
 - null values
 - treatment values
 - discount values
 - purchase logic

- customer IDs
- Generate data quality report

Days 4–6 — Data Engineering

Day 4

- Build reusable data loader
- Add configurable dataset path
- Add configurable batch size
- Add logging

Day 5

- Create database views:
 - customer summary
 - campaign summary
 - treatment summary
 - revenue summary

Day 6

- Add data refresh workflow
- Test fresh database creation from zero
- Document database setup

Days 7–9 — Quality & Performance

Day 7

- Add indexes
- Test query performance
- Validate joins

Day 8

- Create export scripts
- Generate standard model-input dataset
- Generate standard dashboard dataset

Day 9

- Create demo dataset
- Create small test dataset
- Create production-size synthetic dataset

Days 10–12 — Reproducibility

Day 10

- One-command database setup

Day 11

- One-command data refresh

Day 12

- Final data validation
- Backup/export
- Documentation

Days 13–15

- Integration testing
- Fix database issues
- Support final demo
- Freeze schema

Final deliverables

```
database/  
├─ schema.sql  
├─ seed.sql  
├─ migrations/  
└─ views.sql  
  
scripts/  
├─ load_data.py  
├─ validate_data.py  
└─ export_data.py  
  
docs/  
└─ database.md
```

Person 1 does NOT need to wait for Persons 2–5.

5. Person 2 — Causal ML

Objective

Build the complete causal inference pipeline independently using the **frozen dataset contract**.

The original role includes treatment/outcome/confounder definition, causal modeling, ITE estimation and validation.

Important change

Person 2 should initially use:

```
sample_customer_data.csv
```

instead of waiting for Person 1's PostgreSQL database.

Days 1–3 — Causal Foundation

Day 1

- Define:
 - treatment
 - outcome
 - confounders
 - covariates
- Document causal assumptions
- Create model configuration

Day 2

- Create preprocessing pipeline
- Handle:
 - missing values
 - categorical variables
 - numerical features
 - scaling/encoding

Day 3

- Implement first causal model
- Generate preliminary treatment effects
- Create synthetic ground-truth comparison

Days 4–6 — ITE Pipeline

Day 4

- Implement DML/causal estimation
- Train nuisance models

Day 5

- Generate customer-level ITE
- Generate:
 - baseline probability
 - treatment probability
- ITE

Day 6

- Save output to standard JSON/CSV format
- Validate output schema

Days 7–9 — Model Quality

Day 7

- Model tuning
- Robustness checks

Day 8

- Add diagnostics
- Add treatment-effect distribution analysis

Day 9

- Compare estimated effects against synthetic ground truth
- Create model-quality report

Days 10–12 — Productionization

Day 10

- Make pipeline executable with one command

Day 11

- Add configuration file

Day 12

- Finalize model output contract

Days 13–15

- Integration test
- Performance tuning
- Final model report

Final deliverables

```
causal_ml/  
├─ preprocessing.py  
├─ model.py  
├─ train.py  
├─ predict.py  
├─ diagnostics.py  
└─ config.yaml  
  
outputs/  
├─ causal_predictions.csv  
└─ causal_summary.json
```

Person 2 never needs to access Person 1's database during development.

At integration time, the real database becomes another input source.

6. Person 3 — Uplift + Optimization

Objective

Build the complete decision/optimization layer using a **pre-generated causal-output fixture**.

The original role covers uplift segmentation, Qini/AUUC, ROI, discount recommendations and budget optimization.

Critical change

Create:

```
sample_causal_predictions.json
```

on Day 1.

It contains realistic ITE values.

Person 3 can therefore start immediately.

Days 1–3 — Uplift Analytics

Day 1

- Define uplift segments:
- persuadable
- sure thing
- lost cause
- sleeping dog

Day 2

- Implement customer ranking
- Implement uplift segmentation

Day 3

- Implement:
- Qini
- AUUC
- incremental conversions
- ROI

Days 4–6 — Optimization

Day 4

- Build discount allocation logic

Day 5

- Implement budget constraints

Example:

```
Budget = ₹100,000
```

```
Customers
↓
Expected incremental value
↓
Discount cost
↓
Expected profit
↓
Select optimal customers
```

Day 6

- Generate recommendation table

Days 7-9 — Scenario Analysis

Day 7

Test:

```
Budget = 25K
Budget = 50K
Budget = 100K
Budget = 250K
```

Day 8

Create scenario comparison.

Day 9

Compare:

```
Random targeting
vs
Causal targeting
```

Days 10-12 — Productionization

Day 10

- Create reusable optimization module

Day 11

- Create recommendation export

Day 12

- Finalize analytics output schema

Days 13–15

- Integration testing
- Validate against real causal output
- Final optimization tuning

Final deliverables

```
uplift/  
├─ segmentation.py  
├─ metrics.py  
├─ qini.py  
├─ auuc.py  
└─ optimization.py  
  
outputs/  
├─ uplift_results.json  
├─ recommendations.json  
└─ scenario_comparison.json
```

Person 3 does not wait for Person 2.

7. Person 4 — Backend API

Objective

Build the complete FastAPI backend independently using **mock repository data**.

The original plan requires FastAPI, PostgreSQL integration, model outputs, validation, testing and Swagger documentation.

Critical change

Person 4 initially does NOT connect to the real database or ML models.

Instead:

```
Mock Repository
  ↓
Service Layer
  ↓
FastAPI
```

Later:

```
PostgreSQL / ML outputs
  ↓
Repository
  ↓
Service Layer
  ↓
FastAPI
```

The API contract doesn't change.

Days 1–3 — API Foundation

Day 1

- Create FastAPI project
- Setup configuration
- Setup logging
- Setup error handling

Day 2

Create all routes:

```
/health
/customers
/summary
/causal-summary
/ite
/uplift
/recommendations
/optimize
```

Day 3

- Create Pydantic request/response models
- Create mock repository

Days 4–6 — API Implementation

Day 4

Implement:

```
/health  
/customers  
/summary
```

using mock data.

Day 5

Implement:

```
/causal-summary  
/ite
```

using mock causal output.

Day 6

Implement:

```
/uplift  
/recommendations  
/optimize
```

using mock analytics output.

Days 7–9 — API Quality

Day 7

- Validation
- Error handling

Day 8

- Unit tests
- API tests

Day 9

- Swagger documentation
- Example payloads

Days 10–12 — Repository Integration

Day 10

Create database repository interface.

Day 11

Create ML/analytics repository interfaces.

Day 12

Replace mock repository with real implementations.

The API layer itself should not require major changes.

Days 13–15

- End-to-end testing
- Performance testing
- Deployment preparation

Final deliverables

```
backend/  
├─ api/  
├─ schemas/  
├─ services/  
├─ repositories/  
├─ models/  
├─ tests/  
└─ main.py
```

Person 4 can complete almost the entire backend without waiting for anyone.

8. Person 5 — Frontend + DevOps

Objective

Build the complete dashboard independently using **static/mock API responses**.

The original role includes the Next.js dashboard, multiple analytical pages, charts, API integration and Docker/deployment.

Critical change

Person 5 should NOT wait for Person 4's API.

Create:

```
mock-api/  
├─ customers.json  
├─ summary.json  
├─ causal-summary.json  
├─ ite.json  
├─ uplift.json  
├─ recommendations.json  
└─ optimize.json
```

The frontend uses these files initially.

Later:

```
Mock API  
↓  
Real API
```

The UI components don't need to change.

Days 1-3 — UI Foundation

Day 1

- Setup Next.js
- Setup project structure
- Setup theme

- Setup routing

Day 2

Create:

- Sidebar
- Header
- Navigation
- Layout
- Cards

Day 3

Create reusable:

- Table
- Chart container
- Metric card
- Filter
- Pagination
- Loading state
- Error state

Days 4–6 — Dashboard

Day 4

Build:

Overview

Day 5

Build:

Causal Insights

Day 6

Build:

Uplift Analysis

Days 7-9 — Remaining Pages

Day 7

Build:

Optimization

Day 8

Build:

Recommendations

Day 9

Add:

- Filters
- Search
- Customer details
- Charts

Required charts include treatment distribution, ITE distribution, Qini curve and uplift segments.

Days 10-12 — API + Docker

Day 10

Replace mock API with real backend.

Day 11

Create:

Dockerfile
docker-compose.yml

Day 12

Make:

```
docker compose up
```

run the complete application.

Days 13–15

- UI polish
- Responsive testing
- Browser testing
- Demo flow
- Deployment testing

Final deliverables

```
frontend/  
├─ app/  
├─ components/  
├─ services/  
├─ charts/  
├─ mocks/  
└─ Dockerfile  
  
docker-compose.yml
```

Person 5 can build approximately 90% of the frontend before Person 4's backend is ready.

9. Shared Contract Repository

Create one small shared folder that everyone can access:

```
contracts/  
├─  
├─ dataset_schema.json  
├─ causal_output.json  
├─ uplift_output.json  
├─ recommendation_output.json  
├─ api_schema.yaml  
└─ sample_data/  
    ├─ customers.json  
    └─ causal.json
```

```
|— uplift.json
|— recommendations.json
```

This becomes the **single source of truth**.

Nobody should change the contract without notifying the team.

10. Mock-First Development Rule

This is the most important change.

Person 1

Works with:

```
CSV → PostgreSQL
```

Person 2

Works with:

```
Sample CSV → Causal Model → Sample Output
```

Person 3

Works with:

```
Sample Causal Output → Uplift → Optimization
```

Person 4

Works with:

```
Mock Repository → FastAPI
```

Person 5

Works with:

Mock API → Next.js

Therefore everyone can work simultaneously.

11. New 15-Day Parallel Schedule

Day	P1 Data/DB	P2 Causal ML	P3 Uplift/ Optimization	P4 Backend	P5 Frontend/ DevOps
1	DB/schema	Causal definition	Metrics definition	API structure	Next.js setup
2	Load data	Preprocessing	Ranking	API schemas	Layout
3	Validation	First model	Segmentation	Mock endpoints	Components
4	ETL	DML	Qini/AUUC	Customer APIs	Overview
5	Views	ITE	Optimization	Summary APIs	Causal page
6	Data quality	Output	Recommendations	Causal APIs	Uplift page
7	Indexes	Tuning	Budget scenarios	Validation	Optimization page
8	Export	Diagnostics	Scenario comparison	Tests	Recommendations
9	Demo dataset	Ground truth	Random vs causal	Swagger	Charts/filters
10	Reproducibility	Pipeline	Optimization final	Repository	Real API connection
11	Refresh workflow	DB output adapter	Recommendation export	DB integration	Docker
12	Final validation	Final report	Final report	Integration	One-command run
13	Integration	Integration	Integration	E2E tests	E2E tests
14	Bug fixing	Bug fixing	Bug fixing	Bug fixing	UI/deployment fixes
15	Demo	Demo	Demo	Demo	Final presentation

12. Dependency Matrix

The target should be:

Person	Can start independently?	Can finish independently?	External dependency
Person 1	✓	✓	None
Person 2	✓	✓	Dataset contract only
Person 3	✓	✓	Causal-output contract only
Person 4	✓	✓	API contract only
Person 5	✓	✓	API contract only

The only real dependency is at **integration time**.

13. Integration Should Happen Only at the End

Instead of integrating continuously:

Day 1
P1 → P2 → P3 → P4 → P5

do this:

Days 1-12

P1 _____
P2 _____
P3 _____
P4 _____
P5 _____

↓

Days 13-15

Integration
↓
Bug fixing
↓
Demo

This dramatically reduces blocking.

14. Definition of Done for Each Person

Person 1

Done when:

```
Database can be created from scratch
      +
Data can be loaded automatically
      +
Validation passes
      +
Required queries work
```

Person 2

Done when:

```
Model runs with one command
      +
ITE generated
      +
Output matches agreed schema
      +
Diagnostics generated
```

Person 3

Done when:

```
Causal output can be consumed
      +
Uplift calculated
      +
Qini/AUUC calculated
      +
Budget optimization works
      +
Recommendations generated
```

Person 4

Done when:

```
All API endpoints work
      +
Mock data works
      +
Real repositories can be plugged in
      +
Tests pass
      +
Swagger available
```

Person 5

Done when:

```
All dashboard pages work
      +
Mock data works
      +
Charts work
      +
Filters work
      +
Real API can be switched on
      +
Docker works
```

15. Rules to Prevent Dependency Problems

Rule 1 — Never wait for another person

If the real dependency isn't available:

use mock/sample data.

Rule 2 — Contracts are more important than implementation

Everyone agrees on:

Input schema
Output schema
Field names
Data types
API response structure

Implementation can change independently.

Rule 3 — No direct database dependency during development

Person 2, 3, 4 and 5 should not require Person 1's live PostgreSQL database.

Rule 4 — No direct ML dependency during development

Person 3 works with sample causal results.

Person 4 works with sample API results.

Person 5 works with sample API responses.

Rule 5 — Integration is a separate phase

Days 13–15 are explicitly reserved for:

- Integration
- E2E testing
- Bug fixing
- Performance adjustments
- Demo preparation

The original plan already reserves these final three days for team-wide integration and polish.

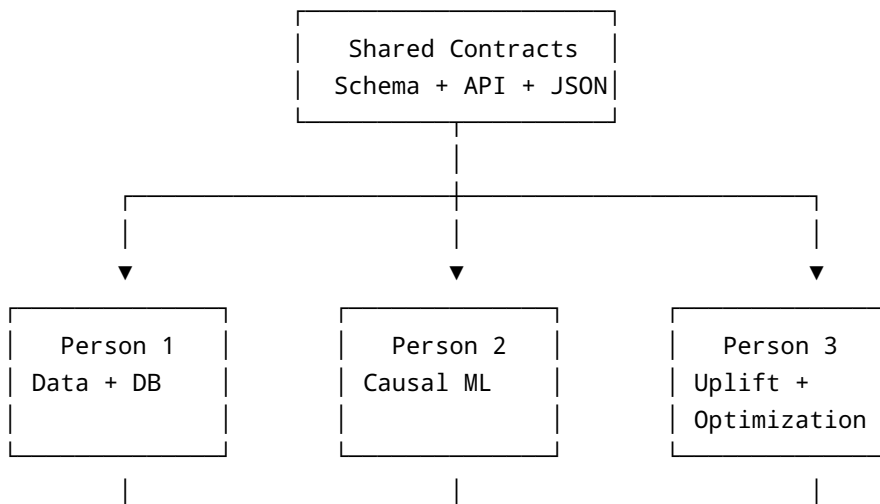
16. Recommended Git Structure

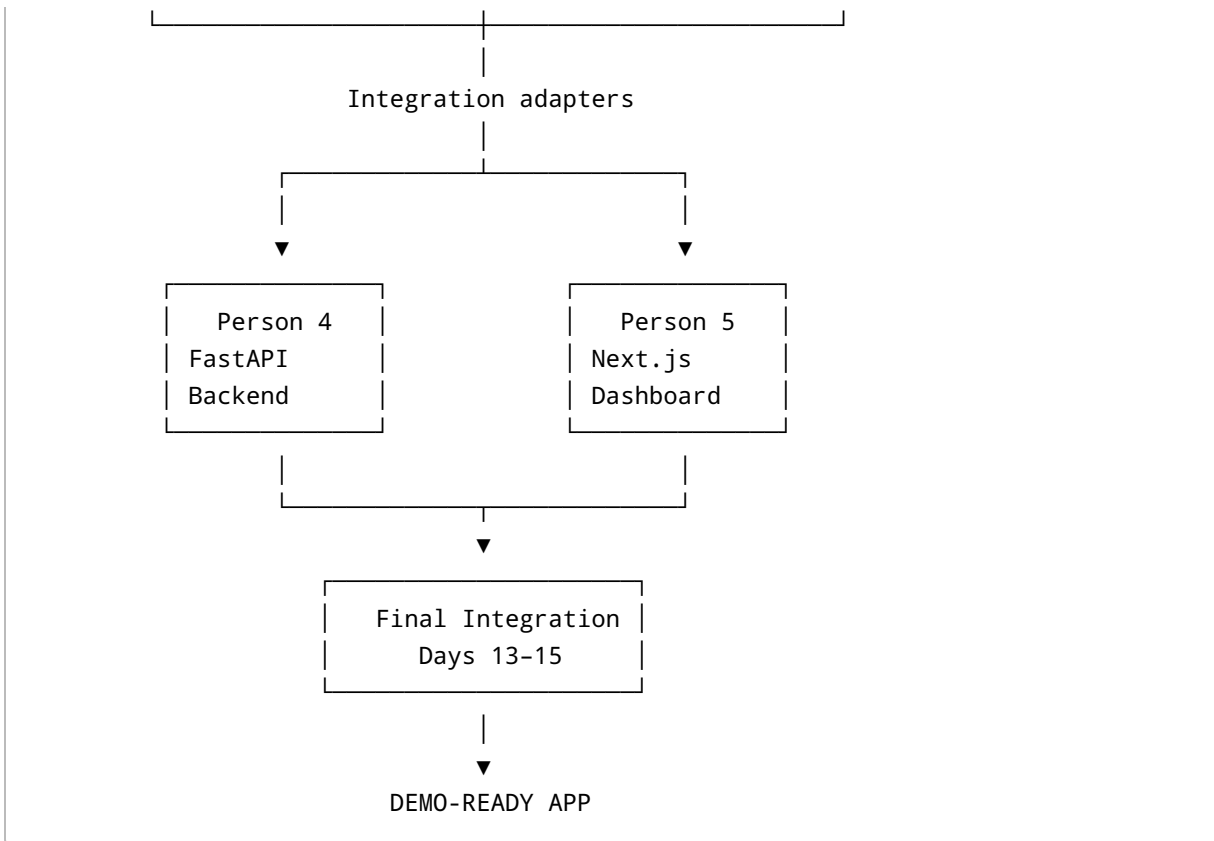
Use separate ownership areas:

```
econocausal/  
├── contracts/  
├── data/  
│   └── person1/  
├── causal_ml/  
│   └── person2/  
├── uplift/  
│   └── person3/  
├── backend/  
│   └── person4/  
├── frontend/  
│   └── person5/  
├── sample_data/  
├── docker-compose.yml  
└── README.md
```

Each person can work primarily inside their own module.

17. Final Architecture





18. Bottom Line

This version is **much safer than the original plan**.

The original plan is technically organized, but its architecture creates a cascading dependency: data → causal ML → optimization → backend → frontend.

The redesigned plan changes that to:

Contract → Parallel development → Adapters → Integration

That means if **Person 2 is delayed**, Person 3 can continue with the sample causal output.

If **Person 4 is delayed**, Person 5 can continue with mock APIs.

If **Person 1 is delayed**, Persons 2–5 can still continue using the frozen sample dataset/contracts.

This gives you a much better chance of having **all five people productive throughout the 15 days instead of people sitting idle waiting for upstream work**.